Convolution Based Networks — Complete Guide

Definitions, Mathematics, Code, and Visualizations

Generated June 2026

Contents

Convolution Based Networks (Mostly for Images)

A complete reference covering the Convolutional Neural Network and its major descendants: LeNet, AlexNet, VGG, GoogLeNet (Inception), ResNet, DenseNet, MobileNet, EfficientNet, U-Net, Mask R-CNN, YOLO, Faster R-CNN, and RetinaNet.

Each section contains: **Definition**, **Intuition**, **Mathematics**, **Architecture Diagram / Graph**, **Worked Example**, and **Complete Runnable Code (PyTorch)**.

1. Convolutional Neural Network (CNN) — The Foundation

1.1 Definition

A **Convolutional Neural Network (CNN)** is a class of deep neural networks designed primarily to process data with a grid-like topology, such as images (2D grid of pixels), audio spectrograms, or video (3D grid). Instead of connecting every input pixel to every neuron (as a fully connected network does), a CNN uses small learnable filters (**kernels**) that slide across the input and detect local patterns such as edges, textures, and shapes. As the network goes deeper, these local patterns are combined into increasingly abstract concepts (edges → textures → parts → objects).

1.2 Why CNNs Instead of Plain Neural Networks?

Property	Fully Connected Network	CNN
Parameters for a 224x224x3 image with 1000 hidden units	~150 billion	A few thousand per filter
Spatial structure	Ignored (image flattened)	Preserved
Translation invariance	None	Built-in (same filter reused everywhere)
Overfitting risk	Very high	Lower (weight sharing)

1.3 Core Mathematical Operation: Convolution

For a 2D image I and a kernel K of size $k \times k$, the convolution (in deep learning, actually **cross-correlation**) at output position (i, j) is:

$$S(i,j) = (I * K)(i,j) = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} I(i+m,\, j+n) \cdot K(m,n) + b$$

where b is a bias term. The output feature map size for an input of size $W \times H$, kernel size k, stride s, and padding p is:

$$W_{out} = \left\lfloor \frac{W - k + 2p}{s} \right\rfloor + 1, \qquad H_{out} = \left\lfloor \frac{H - k + 2p}{s} \right\rfloor + 1$$

Worked numeric example. Input size $W = H = 32$, kernel $k = 3$, stride $s = 1$, padding $p = 1$:

$$W_{out} = \frac{32 - 3 + 2(1)}{1} + 1 = \frac{31}{1} + 1 = 32$$

So a 3x3 kernel with padding 1 preserves spatial size — the classic "same padding" convolution used throughout VGG, ResNet, etc.

1.4 Receptive Field

The **receptive field** of a unit is the region of the original input image that influences its value. Stacking 3x3 convolutions grows the receptive field linearly with depth:

$$RF_l = RF_{l-1} + (k-1) \cdot \prod_{i=1}^{l-1} s_i$$

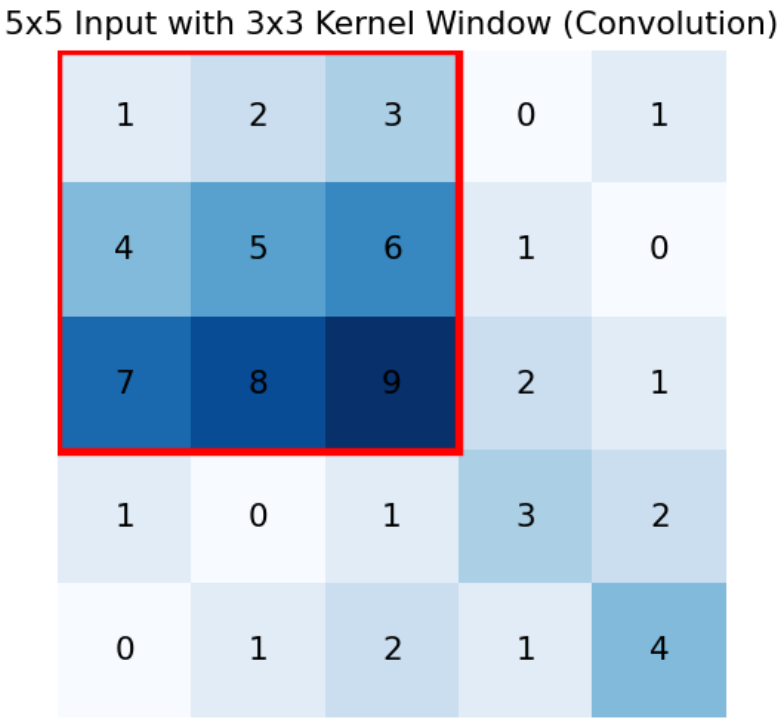

Figure 1: 5x5 input image with a 3x3 sliding kernel window highlighted in red. The kernel slides across the image, computing a weighted sum at every position.

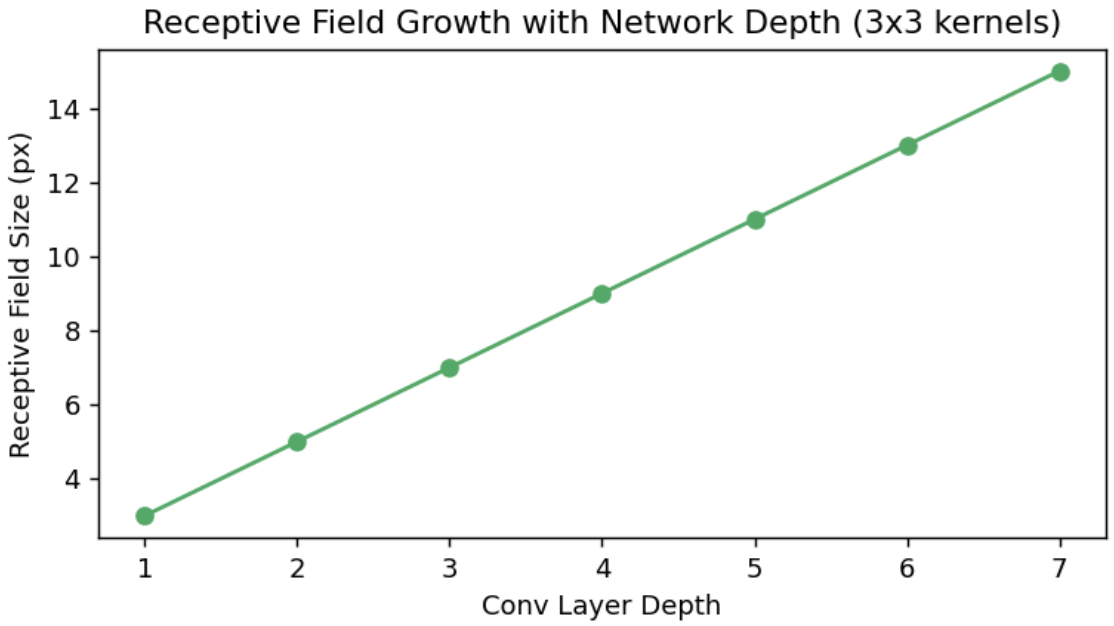

Figure 2: Receptive field size growing linearly as more 3x3 convolution layers are stacked.

1.5 Pooling

Pooling reduces spatial resolution and adds a small amount of translation invariance. Max-pooling over a $k \times k$ window:

$$P(i,j) = \max_{0 \le m,n < k} I(i \cdot s + m,\ j \cdot s + n)$$

Average pooling replaces the max with the mean.

1.6 Activation: ReLU

$$\mathrm{ReLU}(x) = \max(0, x)$$

ReLU is preferred over sigmoid/tanh in CNNs because its gradient does not vanish for $x > 0$, which lets gradients propagate through very deep stacks of layers.

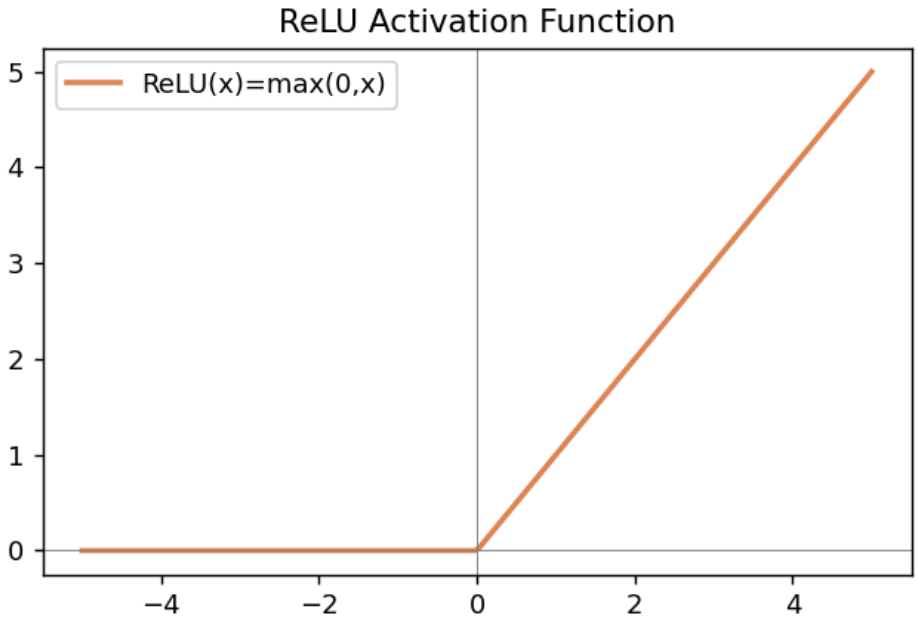

Figure 3: Plot of the ReLU activation function: f(x) = max(0, x).

1.7 Training: Backpropagation Through Convolution

The network is trained by minimizing a loss (e.g. cross-entropy for classification):

$$\mathcal{L} = -\sum_{c=1}^{C} y_c \log(\hat{y}_c)$$

Gradients of the loss with respect to kernel weights are computed via the chain rule; because convolution is a linear operation, its gradient is itself a convolution (of the upstream gradient with a flipped kernel). Weights are updated with an optimizer such as SGD or Adam:

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$$

1.8 Complete Minimal CNN — Full Code (PyTorch)

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

# ---- 1. Define a simple CNN for MNIST-like 1-channel 28x28 images ----
class SimpleCNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.conv1 = nn.Conv2d(in_channels=1, out_channels=16,
                               kernel_size=3, stride=1, padding=1)
        self.conv2 = nn.Conv2d(in_channels=16, out_channels=32,
                               kernel_size=3, stride=1, padding=1)
        self.pool  = nn.MaxPool2d(kernel_size=2, stride=2)
        self.fc1   = nn.Linear(32 * 7 * 7, 128)
        self.fc2   = nn.Linear(128, num_classes)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))   # 28x28 -> 14x14
        x = self.pool(F.relu(self.conv2(x)))   # 14x14 -> 7x7
        x = torch.flatten(x, 1)
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x

# ---- 2. Data ----
transform = transforms.Compose([transforms.ToTensor()])
train_data = datasets.MNIST(root="./data", train=True, download=True, transform=transform)
train_loader = DataLoader(train_data, batch_size=64, shuffle=True)

# ---- 3. Train ----
device = "cuda" if torch.cuda.is_available() else "cpu"
model = SimpleCNN().to(device)
optimizer = optim.Adam(model.parameters(), lr=1e-3)
criterion = nn.CrossEntropyLoss()

for epoch in range(3):
    model.train()
    running_loss = 0.0
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()
    print(f"Epoch {epoch+1}: loss = {running_loss/len(train_loader):.4f}")
```

This single block is a complete, runnable training loop: it builds the model, loads data, computes the loss,

backpropagates, and updates weights. Every CNN below follows the same skeleton — only the `Module` architecture changes.

2. LeNet-5 (1998)

2.1 Definition

LeNet-5, designed by Yann LeCun, is the first practical CNN, originally built to recognize handwritten digits (the MNIST dataset) for reading bank cheques. It established the template every later CNN follows: `conv -> pool -> conv -> pool -> fully connected -> output`.

2.2 Architecture

Layer	Type	Output Shape	Params
Input	Image	32x32x1	-
C1	Conv 5x5, 6 filters	28x28x6	156
S2	AvgPool 2x2	14x14x6	-
C3	Conv 5x5, 16 filters	10x10x16	2416
S4	AvgPool 2x2	5x5x16	-
C5	Conv 5x5, 120 filters (FC-like)	1x1x120	48120
F6	Fully Connected	84	10164
Output	Fully Connected + Softmax	10	850

Total parameters: about 60,000 — tiny by modern standards but groundbreaking in 1998.

2.3 Math

LeNet uses `tanh` activations (ReLU came later):

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

The final layer applies softmax for 10-class digit probabilities:

$$\hat{y}_c = \frac{e^{z_c}}{\sum_{j=1}^{10} e^{z_j}}$$

2.4 Full Code (PyTorch)

```
import torch
import torch.nn as nn

class LeNet5(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 6, kernel_size=5)      # 32x32 -> 28x28
        self.conv2 = nn.Conv2d(6, 16, kernel_size=5)     # 14x14 -> 10x10
        self.pool  = nn.AvgPool2d(2, 2)
        self.fc1   = nn.Linear(16 * 5 * 5, 120)
        self.fc2   = nn.Linear(120, 84)
        self.fc3   = nn.Linear(84, num_classes)

    def forward(self, x):
        x = self.pool(torch.tanh(self.conv1(x)))
        x = self.pool(torch.tanh(self.conv2(x)))
```

```
        x = torch.flatten(x, 1)
        x = torch.tanh(self.fc1(x))
        x = torch.tanh(self.fc2(x))
        return self.fc3(x)

model = LeNet5()
sample = torch.randn(1, 1, 32, 32)
print(model(sample).shape)    # torch.Size([1, 10])
```

3. AlexNet (2012)

3.1 Definition

AlexNet, by Krizhevsky, Sutskever and Hinton, won the ImageNet 2012 competition and is widely credited with starting the deep-learning revolution in computer vision. It scaled LeNet's idea to 8 layers, used ReLU, dropout, GPU training, and data augmentation.

3.2 Key Innovations

- **ReLU activation** instead of tanh/sigmoid, much faster training.
- **Dropout** (p=0.5) in fully connected layers to fight overfitting.
- **Local Response Normalization** (LRN).
- **Overlapping max-pooling**.
- Trained on **2 GPUs** in parallel.

3.3 Architecture (simplified, single-GPU equivalent)

Layer	Type	Kernel/Stride	Output
Input	Image	-	224x224x3
Conv1	Conv 11x11, 96 filters	s=4	55x55x96
Pool1	MaxPool 3x3	s=2	27x27x96
Conv2	Conv 5x5, 256 filters	s=1,p=2	27x27x256
Pool2	MaxPool 3x3	s=2	13x13x256
Conv3	Conv 3x3, 384 filters	s=1,p=1	13x13x384
Conv4	Conv 3x3, 384 filters	s=1,p=1	13x13x384
Conv5	Conv 3x3, 256 filters	s=1,p=1	13x13x256
Pool5	MaxPool 3x3	s=2	6x6x256
FC6	Fully Connected	-	4096
FC7	Fully Connected	-	4096
FC8	Fully Connected + Softmax	-	1000

Total parameters: about 60 million.

3.4 Math: Dropout

During training, each unit is zeroed with probability p and surviving units are scaled by $1/(1-p)$:

$$\tilde{h}_i = \frac{m_i}{1-p} h_i, \qquad m_i \sim \text{Bernoulli}(1-p)$$

This prevents co-adaptation of neurons and approximates training an ensemble of sub-networks.

3.5 Full Code (PyTorch)

```
import torch
import torch.nn as nn

class AlexNet(nn.Module):
    def __init__(self, num_classes=1000):
        super().__init__()
        self.features = nn.Sequential(
```

```
            nn.Conv2d(3, 96, kernel_size=11, stride=4, padding=2),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),

            nn.Conv2d(96, 256, kernel_size=5, padding=2),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),

            nn.Conv2d(256, 384, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(384, 384, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(384, 256, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
        )
        self.classifier = nn.Sequential(
            nn.Dropout(0.5),
            nn.Linear(256 * 6 * 6, 4096),
            nn.ReLU(inplace=True),
            nn.Dropout(0.5),
            nn.Linear(4096, 4096),
            nn.ReLU(inplace=True),
            nn.Linear(4096, num_classes),
        )

    def forward(self, x):
        x = self.features(x)
        x = torch.flatten(x, 1)
        return self.classifier(x)

model = AlexNet()
print(model(torch.randn(1, 3, 224, 224)).shape)  # torch.Size([1, 1000])
```

4. VGG (2014)

4.1 Definition

VGGNet (Simonyan & Zisserman, Oxford) showed that depth, achieved by stacking many small **3x3** convolutions, matters more than using large filters. VGG-16 has 16 weight layers; VGG-19 has 19.

4.2 Why 3x3 Kernels?

Two stacked 3x3 convolutions have the same receptive field as one 5x5 convolution, but with fewer parameters and an extra non-linearity:

$$\underbrace{2 \times (3 \times 3 \times C^2)}_{\text{two 3x3 convs}} = 18C^2 \quad < \quad \underbrace{1 \times (5 \times 5 \times C^2)}_{\text{one 5x5 conv}} = 25C^2$$

Three stacked 3x3 convs match a 7x7 conv's receptive field with even greater parameter savings: $27C^2$ vs $49C^2$.

4.3 Architecture (VGG-16)

```
Input 224x224x3
[Conv3-64]  x2  -> MaxPool   => 112x112x64
[Conv3-128] x2  -> MaxPool   => 56x56x128
[Conv3-256] x3  -> MaxPool   => 28x28x256
[Conv3-512] x3  -> MaxPool   => 14x14x512
[Conv3-512] x3  -> MaxPool   => 7x7x512
Flatten -> FC-4096 -> FC-4096 -> FC-1000 (softmax)
```

Total parameters: about 138 million (mostly in the fully connected layers).

4.4 Full Code (PyTorch)

```python
import torch
import torch.nn as nn

def vgg_block(in_c, out_c, n_convs):
    layers = []
    for i in range(n_convs):
        layers += [nn.Conv2d(in_c if i == 0 else out_c, out_c,
                             kernel_size=3, padding=1),
                   nn.ReLU(inplace=True)]
    layers.append(nn.MaxPool2d(kernel_size=2, stride=2))
    return nn.Sequential(*layers)

class VGG16(nn.Module):
    def __init__(self, num_classes=1000):
        super().__init__()
        self.features = nn.Sequential(
            vgg_block(3,   64, 2),
            vgg_block(64,  128, 2),
            vgg_block(128, 256, 3),
            vgg_block(256, 512, 3),
            vgg_block(512, 512, 3),
        )
```

```
        self.classifier = nn.Sequential(
            nn.Linear(512 * 7 * 7, 4096), nn.ReLU(inplace=True), nn.Dropout(0.5),
            nn.Linear(4096, 4096),        nn.ReLU(inplace=True), nn.Dropout(0.5),
            nn.Linear(4096, num_classes),
        )

    def forward(self, x):
        x = self.features(x)
        x = torch.flatten(x, 1)
        return self.classifier(x)

model = VGG16()
print(model(torch.randn(1, 3, 224, 224)).shape)  # torch.Size([1, 1000])
```

5. GoogLeNet / Inception (2014)

5.1 Definition

GoogLeNet (Szegedy et al.) introduced the **Inception module**: instead of choosing one filter size, run several filter sizes (1x1, 3x3, 5x5) and a pooling branch **in parallel** on the same input, then concatenate their outputs along the channel dimension. This captures multi-scale features efficiently and won ImageNet 2014 with only 5 million parameters (vs VGG's 138M).

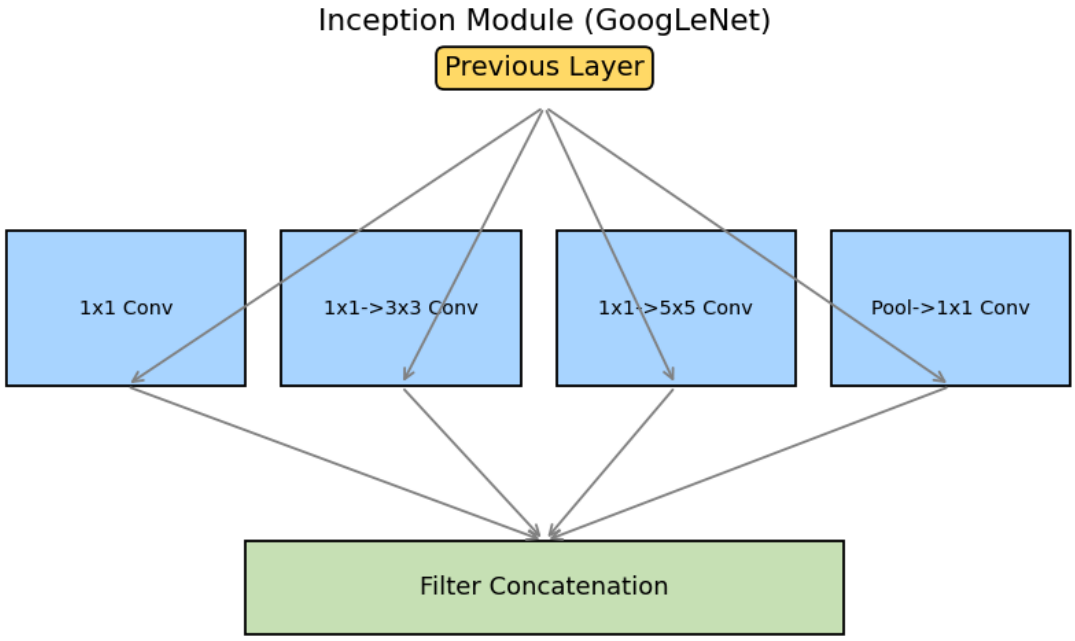

Figure 4: The Inception module: four parallel branches process the same input at different receptive-field scales and are concatenated.

5.2 The 1x1 Convolution Trick

A 1x1 convolution before the expensive 3x3/5x5 branches reduces channel depth cheaply (a "bottleneck"), cutting computation drastically:

$$\text{Cost}_{\text{naive}} = H \cdot W \cdot C_{in} \cdot C_{out} \cdot k^2$$

$$\text{Cost}_{\text{bottleneck}} = H \cdot W \cdot C_{in} \cdot C_{mid} + H \cdot W \cdot C_{mid} \cdot C_{out} \cdot k^2, \quad C_{mid} \ll C_{in}$$

5.3 Auxiliary Classifiers

GoogLeNet attaches two extra softmax classifiers at intermediate depths during training (discarded at inference) to inject additional gradient signal and combat vanishing gradients in a 22-layer network:

$$\mathcal{L}_{total} = \mathcal{L}_{final} + 0.3 \cdot \mathcal{L}_{aux1} + 0.3 \cdot \mathcal{L}_{aux2}$$

5.4 Full Code (PyTorch) — Inception Module + Mini GoogLeNet

```
import torch
import torch.nn as nn

class InceptionModule(nn.Module):
    def __init__(self, in_c, c1, c3_red, c3, c5_red, c5, pool_proj):
        super().__init__()
        self.branch1 = nn.Sequential(
            nn.Conv2d(in_c, c1, kernel_size=1), nn.ReLU(inplace=True))
        self.branch2 = nn.Sequential(
            nn.Conv2d(in_c, c3_red, kernel_size=1), nn.ReLU(inplace=True),
            nn.Conv2d(c3_red, c3, kernel_size=3, padding=1), nn.ReLU(inplace=True))
        self.branch3 = nn.Sequential(
            nn.Conv2d(in_c, c5_red, kernel_size=1), nn.ReLU(inplace=True),
            nn.Conv2d(c5_red, c5, kernel_size=5, padding=2), nn.ReLU(inplace=True))
        self.branch4 = nn.Sequential(
            nn.MaxPool2d(kernel_size=3, stride=1, padding=1),
            nn.Conv2d(in_c, pool_proj, kernel_size=1), nn.ReLU(inplace=True))

    def forward(self, x):
        b1, b2, b3, b4 = self.branch1(x), self.branch2(x), self.branch3(x), self.branch4(x)
        return torch.cat([b1, b2, b3, b4], dim=1)  # concatenate along channels

class MiniGoogLeNet(nn.Module):
    def __init__(self, num_classes=1000):
        super().__init__()
        self.stem = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=7, stride=2, padding=3), nn.ReLU(inplace=True),
            nn.MaxPool2d(3, stride=2, padding=1),
        )
        self.inception3a = InceptionModule(64, 64, 96, 128, 16, 32, 32)
        self.pool = nn.AdaptiveAvgPool2d((1, 1))
        self.fc = nn.Linear(64 + 128 + 32 + 32, num_classes)

    def forward(self, x):
        x = self.stem(x)
        x = self.inception3a(x)
        x = self.pool(x)
        x = torch.flatten(x, 1)
        return self.fc(x)

model = MiniGoogLeNet()
print(model(torch.randn(1, 3, 224, 224)).shape)  # torch.Size([1, 1000])
```

6. ResNet (2015)

6.1 Definition

ResNet (He et al., Microsoft) solved the **vanishing gradient / degradation problem** that prevented very deep networks (50, 101, 152+ layers) from training well. The fix: **residual (skip) connections** that let a block learn a residual function $F(x)$ added back to its input, instead of learning the full mapping directly.

$$y = F(x, \{W_i\}) + x$$

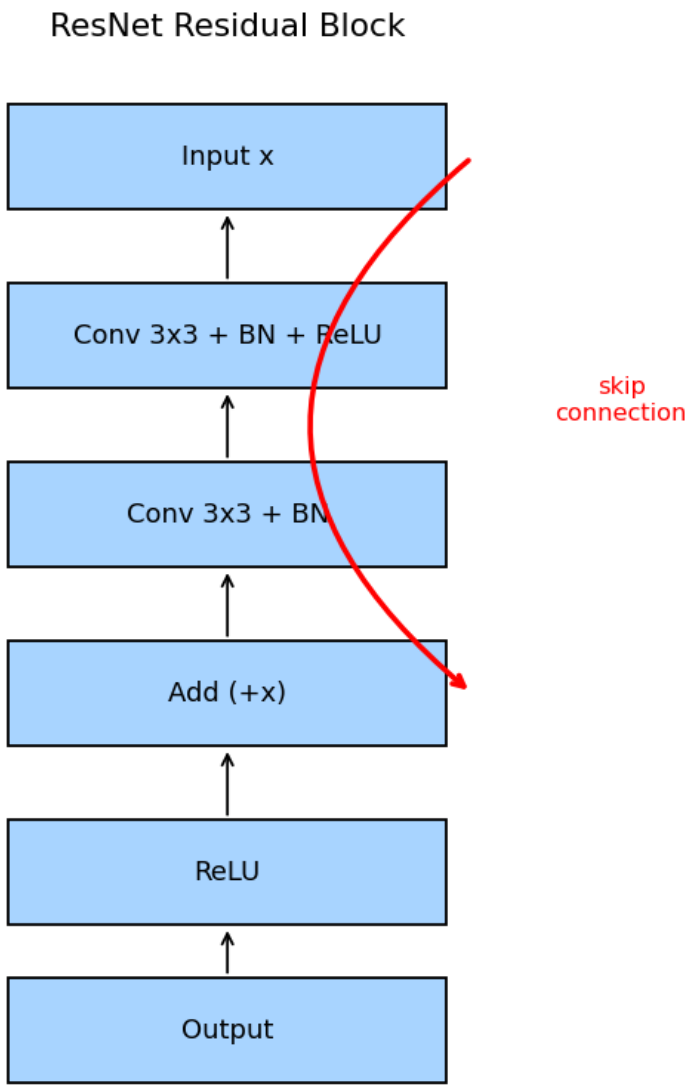

Figure 5: A ResNet residual block: the input x is added back to the output of two stacked convolutions before the final ReLU. The skip connection (red) lets gradients flow directly to earlier layers.

6.2 Why Skip Connections Solve Vanishing Gradients

During backpropagation, the gradient of the loss with respect to an early layer's input includes an identity term:

$$\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial y}\left(1 + \frac{\partial F}{\partial x}\right)$$

The "+1" guarantees a direct, unattenuated gradient path no matter how deep the network is, which is why ResNet-152 trains successfully while a plain 152-layer network does not.

6.3 Bottleneck Block (ResNet-50/101/152)

To keep computation manageable in deeper variants, a 3-conv "bottleneck" is used: 1x1 (reduce channels) → 3x3 (spatial) → 1x1 (restore channels).

6.4 Full Code (PyTorch)

```
import torch
import torch.nn as nn

class BasicBlock(nn.Module):
    """Used in ResNet-18/34."""
    def __init__(self, in_c, out_c, stride=1):
        super().__init__()
        self.conv1 = nn.Conv2d(in_c, out_c, 3, stride, 1, bias=False)
        self.bn1   = nn.BatchNorm2d(out_c)
        self.conv2 = nn.Conv2d(out_c, out_c, 3, 1, 1, bias=False)
        self.bn2   = nn.BatchNorm2d(out_c)
        self.relu  = nn.ReLU(inplace=True)
        self.shortcut = nn.Sequential()
        if stride != 1 or in_c != out_c:
            self.shortcut = nn.Sequential(
                nn.Conv2d(in_c, out_c, 1, stride, bias=False),
                nn.BatchNorm2d(out_c))

    def forward(self, x):
        identity = self.shortcut(x)
        out = self.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        out += identity                # <-- the residual addition
        return self.relu(out)

class ResNet18(nn.Module):
    def __init__(self, num_classes=1000):
        super().__init__()
        self.stem = nn.Sequential(
            nn.Conv2d(3, 64, 7, 2, 3, bias=False), nn.BatchNorm2d(64),
            nn.ReLU(inplace=True), nn.MaxPool2d(3, 2, 1))
        self.layer1 = self._make_layer(64, 64, 2, stride=1)
        self.layer2 = self._make_layer(64, 128, 2, stride=2)
        self.layer3 = self._make_layer(128, 256, 2, stride=2)
        self.layer4 = self._make_layer(256, 512, 2, stride=2)
        self.avgpool = nn.AdaptiveAvgPool2d((1, 1))
```

```
        self.fc = nn.Linear(512, num_classes)

    def _make_layer(self, in_c, out_c, blocks, stride):
        layers = [BasicBlock(in_c, out_c, stride)]
        for _ in range(1, blocks):
            layers.append(BasicBlock(out_c, out_c, 1))
        return nn.Sequential(*layers)

    def forward(self, x):
        x = self.stem(x)
        x = self.layer1(x); x = self.layer2(x)
        x = self.layer3(x); x = self.layer4(x)
        x = self.avgpool(x)
        x = torch.flatten(x, 1)
        return self.fc(x)

model = ResNet18()
print(model(torch.randn(1, 3, 224, 224)).shape)  # torch.Size([1, 1000])
```

7. DenseNet (2016)

7.1 Definition

DenseNet (Huang et al.) takes the skip-connection idea further: every layer inside a "dense block" receives, as input, the **concatenation** of the feature maps from *all* preceding layers in that block — not just the immediately previous one.

$$x_l = H_l([x_0, x_1, \ldots, x_{l-1}])$$

where $[\cdot]$ denotes channel-wise concatenation and H_l is a composite function (BatchNorm $\rightarrow$ ReLU $\rightarrow$ Conv).

7.2 Why This Helps

- **Feature reuse**: later layers directly access early low-level features.
- **Fewer parameters**: each layer only needs to learn a small **growth rate** k of new feature maps (e.g. $k = 32$), since it can reuse the rest.
- **Stronger gradient flow**: every layer has a direct path back to the loss and to the input.

7.3 Transition Layers

Between dense blocks, a transition layer (1x1 conv + average pool) compresses channels and downsamples spatially, since concatenation alone would make channel counts explode.

7.4 Full Code (PyTorch)

```
import torch
import torch.nn as nn

class DenseLayer(nn.Module):
    def __init__(self, in_c, growth_rate):
        super().__init__()
        self.bn   = nn.BatchNorm2d(in_c)
        self.relu = nn.ReLU(inplace=True)
        self.conv = nn.Conv2d(in_c, growth_rate, kernel_size=3, padding=1, bias=False)

    def forward(self, x):
        out = self.conv(self.relu(self.bn(x)))
        return torch.cat([x, out], dim=1)   # concatenate with ALL previous features

class DenseBlock(nn.Module):
    def __init__(self, in_c, growth_rate, num_layers):
        super().__init__()
        layers = []
        for i in range(num_layers):
            layers.append(DenseLayer(in_c + i * growth_rate, growth_rate))
        self.block = nn.Sequential(*layers)

    def forward(self, x):
        return self.block(x)

class TransitionLayer(nn.Module):
    def __init__(self, in_c, out_c):
        super().__init__()
```

```
        self.bn   = nn.BatchNorm2d(in_c)
        self.conv = nn.Conv2d(in_c, out_c, kernel_size=1, bias=False)
        self.pool = nn.AvgPool2d(2, 2)

    def forward(self, x):
        return self.pool(self.conv(self.bn(x)))

class MiniDenseNet(nn.Module):
    def __init__(self, num_classes=1000, growth_rate=32):
        super().__init__()
        self.stem = nn.Sequential(
            nn.Conv2d(3, 64, 7, 2, 3, bias=False), nn.BatchNorm2d(64),
            nn.ReLU(inplace=True), nn.MaxPool2d(3, 2, 1))
        self.block1 = DenseBlock(64, growth_rate, num_layers=6)
        ch_after_block1 = 64 + 6 * growth_rate
        self.trans1 = TransitionLayer(ch_after_block1, ch_after_block1 // 2)
        self.pool = nn.AdaptiveAvgPool2d((1, 1))
        self.fc = nn.Linear(ch_after_block1 // 2, num_classes)

    def forward(self, x):
        x = self.stem(x)
        x = self.block1(x)
        x = self.trans1(x)
        x = self.pool(x)
        x = torch.flatten(x, 1)
        return self.fc(x)

model = MiniDenseNet()
print(model(torch.randn(1, 3, 224, 224)).shape)  # torch.Size([1, 1000])
```

8. MobileNet (2017)

8.1 Definition

MobileNet (Howard et al., Google) is designed for mobile and embedded devices with limited compute. Its key innovation is **depthwise separable convolution**, which factorizes a standard convolution into two cheaper steps.

8.2 Depthwise Separable Convolution — Math

A standard convolution costs:

$$\text{Cost}_{std} = H \cdot W \cdot C_{in} \cdot C_{out} \cdot k^2$$

Depthwise separable convolution splits this into:

1. **Depthwise conv**: apply one $k \times k$ filter per input channel (no mixing across channels):

$$\text{Cost}_{dw} = H \cdot W \cdot C_{in} \cdot k^2$$

2. **Pointwise conv** (1x1): mix channels:

$$\text{Cost}_{pw} = H \cdot W \cdot C_{in} \cdot C_{out}$$

Total: $\text{Cost}_{dw+pw} = HWC_{in}(k^2 + C_{out})$, giving a speed-up factor of roughly:

$$\frac{\text{Cost}_{std}}{\text{Cost}_{dw+pw}} \approx \frac{1}{C_{out}} + \frac{1}{k^2}$$

For $k = 3, C_{out} = 256$: roughly an **8-9x reduction** in computation.

8.3 Width Multiplier and Resolution Multiplier

MobileNet exposes two hyperparameters to trade accuracy for speed: α (scales channel counts) and ρ (scales input resolution).

8.4 Full Code (PyTorch)

```
import torch
import torch.nn as nn

class DepthwiseSeparableConv(nn.Module):
    def __init__(self, in_c, out_c, stride=1):
        super().__init__()
        self.depthwise = nn.Conv2d(in_c, in_c, kernel_size=3, stride=stride,
                                   padding=1, groups=in_c, bias=False)
        self.pointwise = nn.Conv2d(in_c, out_c, kernel_size=1, bias=False)
        self.bn1 = nn.BatchNorm2d(in_c)
        self.bn2 = nn.BatchNorm2d(out_c)
        self.relu = nn.ReLU6(inplace=True)

    def forward(self, x):
        x = self.relu(self.bn1(self.depthwise(x)))
        x = self.relu(self.bn2(self.pointwise(x)))
        return x
```

```
class MobileNetV1(nn.Module):
    def __init__(self, num_classes=1000, alpha=1.0):
        super().__init__()
        def c(ch): return int(ch * alpha)
        self.model = nn.Sequential(
            nn.Conv2d(3, c(32), 3, 2, 1, bias=False), nn.BatchNorm2d(c(32)), nn.ReLU6(inplace=True),
            DepthwiseSeparableConv(c(32), c(64), 1),
            DepthwiseSeparableConv(c(64), c(128), 2),
            DepthwiseSeparableConv(c(128), c(128), 1),
            DepthwiseSeparableConv(c(128), c(256), 2),
            DepthwiseSeparableConv(c(256), c(256), 1),
            DepthwiseSeparableConv(c(256), c(512), 2),
            nn.AdaptiveAvgPool2d((1, 1)),
        )
        self.fc = nn.Linear(c(512), num_classes)

    def forward(self, x):
        x = self.model(x)
        x = torch.flatten(x, 1)
        return self.fc(x)

model = MobileNetV1()
print(model(torch.randn(1, 3, 224, 224)).shape)  # torch.Size([1, 1000])
```

9. EfficientNet (2019)

9.1 Definition

EfficientNet (Tan & Le, Google) asks: if you have more compute budget, what is the *optimal* way to scale a CNN — wider, deeper, or higher resolution? Prior work scaled just one dimension arbitrarily. EfficientNet introduces **compound scaling**, scaling depth, width, and resolution together using a single coefficient ϕ.

9.2 Compound Scaling — Math

$$\text{depth: } d = \alpha^\phi, \qquad \text{width: } w = \beta^\phi, \qquad \text{resolution: } r = \gamma^\phi$$

subject to $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$ and $\alpha \geq 1, \beta \geq 1, \gamma \geq 1$. The constraint comes from the fact that FLOPs of a conv layer scale as $d \cdot w^2 \cdot r^2$, so fixing the product to 2 means doubling ϕ roughly doubles total FLOPs in a balanced way, found by grid search to be $\alpha \approx 1.2,\ \beta \approx 1.1,\ \gamma \approx 1.15$.

9.3 Backbone: MBConv (Mobile Inverted Bottleneck) + Squeeze-and-Excitation

EfficientNet's base network (B0) uses **MBConv** blocks: 1x1 expand → 3x3 (or 5x5) depthwise conv → **Squeeze-and-Excitation (SE)** channel attention → 1x1 project, with a residual connection. SE re-weights channels by their global importance:

$$s_c = \sigma(W_2\, \delta(W_1\, z_c)), \qquad z_c = \frac{1}{HW} \sum_{i,j} x_{c,i,j}$$

where z_c is the global-average-pooled value for channel c, δ is ReLU, and σ is sigmoid; s_c rescales channel c.

9.4 Full Code (PyTorch) — MBConv Block

```
import torch
import torch.nn as nn

class SqueezeExcite(nn.Module):
    def __init__(self, channels, reduction=4):
        super().__init__()
        self.pool = nn.AdaptiveAvgPool2d(1)
        self.fc1  = nn.Conv2d(channels, channels // reduction, 1)
        self.fc2  = nn.Conv2d(channels // reduction, channels, 1)
        self.relu = nn.ReLU(inplace=True)
        self.sig  = nn.Sigmoid()

    def forward(self, x):
        s = self.pool(x)
        s = self.relu(self.fc1(s))
        s = self.sig(self.fc2(s))
        return x * s    # channel-wise reweighting

class MBConvBlock(nn.Module):
    def __init__(self, in_c, out_c, expand_ratio=6, stride=1, k=3):
        super().__init__()
        mid_c = in_c * expand_ratio
        self.expand = nn.Sequential(
            nn.Conv2d(in_c, mid_c, 1, bias=False), nn.BatchNorm2d(mid_c), nn.SiLU())
        self.dwconv = nn.Sequential(
```

```
            nn.Conv2d(mid_c, mid_c, k, stride, k // 2, groups=mid_c, bias=False),
            nn.BatchNorm2d(mid_c), nn.SiLU())
        self.se = SqueezeExcite(mid_c)
        self.project = nn.Sequential(
            nn.Conv2d(mid_c, out_c, 1, bias=False), nn.BatchNorm2d(out_c))
        self.use_residual = (stride == 1 and in_c == out_c)

    def forward(self, x):
        out = self.expand(x)
        out = self.dwconv(out)
        out = self.se(out)
        out = self.project(out)
        if self.use_residual:
            out = out + x
        return out

model = MBConvBlock(in_c=32, out_c=32, expand_ratio=6, stride=1)
print(model(torch.randn(1, 32, 56, 56)).shape)  # torch.Size([1, 32, 56, 56])
```

10. U-Net (2015)

10.1 Definition

U-Net (Ronneberger et al.) is an encoder-decoder CNN built for **semantic segmentation** — predicting a class label for every pixel (originally for biomedical images). Its "U" shape comes from a contracting encoder path that downsamples and extracts context, a symmetric expanding decoder path that upsamples back to full resolution, and **skip connections** that copy encoder feature maps directly to the corresponding decoder level.

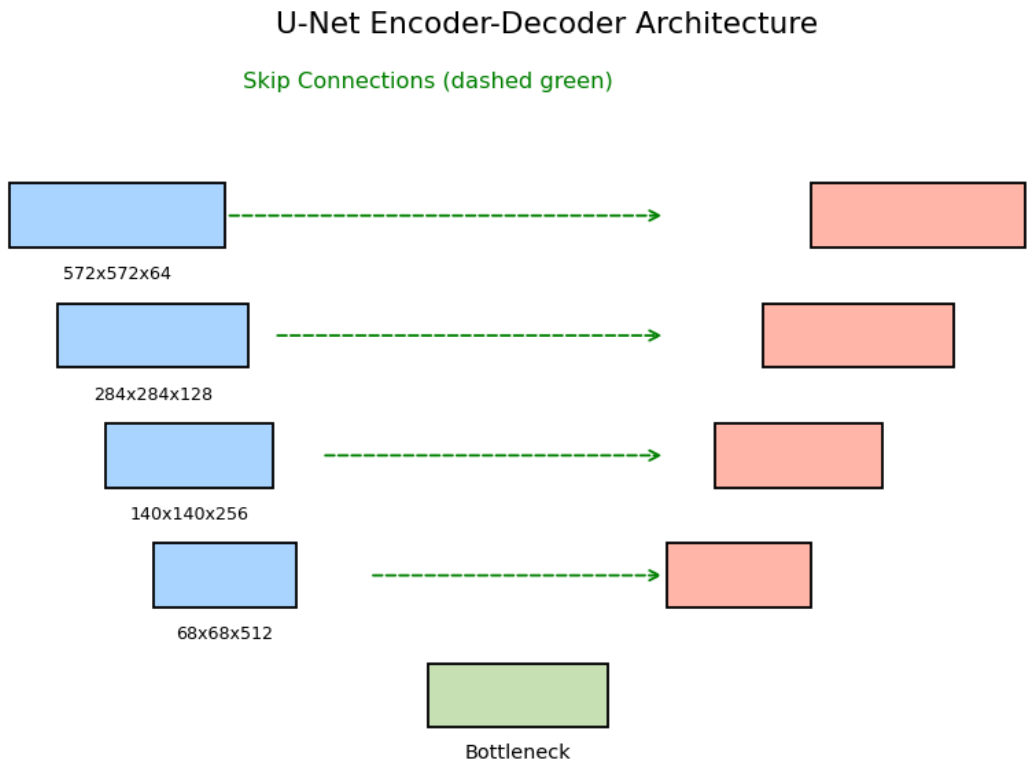

Figure 6: U-Net's encoder (blue, left) progressively downsamples while the decoder (pink, right) upsamples. Skip connections (green dashed) copy fine spatial detail from encoder to decoder at matching resolutions.

10.2 Why Skip Connections Matter Here

Downsampling loses precise spatial location (needed for pixel-accurate masks) even though it gains semantic context. Concatenating the high-resolution encoder features into the decoder lets the network combine "what" (deep semantic features) with "where" (shallow spatial features):

$$D_l = \mathrm{Up}(D_{l+1}) \oplus E_l$$

where $\oplus$ is channel concatenation and E_l is the encoder output at the matching resolution.

10.3 Loss Function

Per-pixel cross-entropy (or Dice loss for class imbalance):

$$\mathcal{L}_{Dice} = 1 - \frac{2\sum_i p_i g_i}{\sum_i p_i^2 + \sum_i g_i^2}$$

where p_i is the predicted probability and g_i the ground-truth label at pixel i.

10.4 Full Code (PyTorch)

```
import torch
import torch.nn as nn

def conv_block(in_c, out_c):
    return nn.Sequential(
        nn.Conv2d(in_c, out_c, 3, padding=1), nn.BatchNorm2d(out_c), nn.ReLU(inplace=True),
        nn.Conv2d(out_c, out_c, 3, padding=1), nn.BatchNorm2d(out_c), nn.ReLU(inplace=True),
    )

class UNet(nn.Module):
    def __init__(self, in_c=3, out_c=1):
        super().__init__()
        self.enc1 = conv_block(in_c, 64)
        self.enc2 = conv_block(64, 128)
        self.enc3 = conv_block(128, 256)
        self.pool = nn.MaxPool2d(2)

        self.bottleneck = conv_block(256, 512)

        self.up3 = nn.ConvTranspose2d(512, 256, 2, stride=2)
        self.dec3 = conv_block(512, 256)
        self.up2 = nn.ConvTranspose2d(256, 128, 2, stride=2)
        self.dec2 = conv_block(256, 128)
        self.up1 = nn.ConvTranspose2d(128, 64, 2, stride=2)
        self.dec1 = conv_block(128, 64)

        self.out_conv = nn.Conv2d(64, out_c, 1)

    def forward(self, x):
        e1 = self.enc1(x)
        e2 = self.enc2(self.pool(e1))
        e3 = self.enc3(self.pool(e2))
        b  = self.bottleneck(self.pool(e3))

        d3 = self.dec3(torch.cat([self.up3(b), e3], dim=1))
        d2 = self.dec2(torch.cat([self.up2(d3), e2], dim=1))
        d1 = self.dec1(torch.cat([self.up1(d2), e1], dim=1))

        return self.out_conv(d1)   # per-pixel logits

model = UNet(in_c=3, out_c=1)
print(model(torch.randn(1, 3, 256, 256)).shape)  # torch.Size([1, 1, 256, 256])
```

11. Faster R-CNN (2015)

11.1 Definition

Faster R-CNN (Ren et al.) is a two-stage **object detector**: stage 1 proposes candidate object regions; stage 2 classifies each region and refines its bounding box. Its key contribution over R-CNN/Fast R-CNN is the **Region Proposal Network (RPN)** — a small CNN that generates region proposals directly from feature maps, replacing slow external algorithms like Selective Search, making the whole pipeline end-to-end trainable and much faster.

11.2 Anchors

At every spatial location of the feature map, the RPN places k **anchor boxes** of different scales/aspect ratios (typically $k = 9$: 3 scales x 3 ratios). For each anchor the RPN predicts: - an **objectness score** (object vs background), and - 4 **bounding-box regression offsets** (t_x, t_y, t_w, t_h).

11.3 Bounding Box Regression — Math

Given anchor (x_a, y_a, w_a, h_a) and ground truth (x^*, y^*, w^*, h^*), targets are parameterized as:

$$t_x = \frac{x^* - x_a}{w_a}, \quad t_y = \frac{y^* - y_a}{h_a}, \quad t_w = \log \frac{w^*}{w_a}, \quad t_h = \log \frac{h^*}{h_a}$$

The RPN regresses to these targets with smooth-L1 loss:

$$\text{smoothL1}(x) = \begin{cases} 0.5x^2 & |x| < 1 \\ |x| - 0.5 & \text{otherwise} \end{cases}$$

Total RPN loss combines classification and regression:

$$\mathcal{L} = \mathcal{L}_{cls} + \lambda \sum_i [p_i^* = 1] \, \text{smoothL1}(t_i - t_i^*)$$

11.4 RoI Pooling / RoI Align

Proposed regions have varying sizes, but the classifier head needs a fixed-size input. **RoI Pooling** divides each proposal into a fixed grid (e.g. 7x7) and max-pools features inside each grid cell.

11.5 Full Code (PyTorch, using torchvision's implementation)

```
import torch
import torchvision
from torchvision.models.detection import fasterrcnn_resnet50_fpn
from torchvision.models.detection.faster_rcnn import FastRCNNPredictor

# Load a Faster R-CNN with a ResNet-50 + FPN backbone, pretrained on COCO
model = fasterrcnn_resnet50_fpn(weights="DEFAULT")

# Replace the classification head for a custom number of classes (e.g. 5 + background)
num_classes = 6
in_features = model.roi_heads.box_predictor.cls_score.in_features
model.roi_heads.box_predictor = FastRCNNPredictor(in_features, num_classes)

model.eval()
```

```
image = torch.rand(3, 480, 640)   # a single RGB image, values in [0,1]
with torch.no_grad():
    predictions = model([image])

print(predictions[0].keys())   # dict_keys(['boxes', 'labels', 'scores'])
print(predictions[0]["boxes"].shape)
```

12. Mask R-CNN (2017)

12.1 Definition

Mask R-CNN (He et al.) extends Faster R-CNN to **instance segmentation**: for every detected object it predicts not just a class and a box, but also a pixel-level binary **mask**. It adds a small FCN ("mask head") in parallel with the existing classification and box- regression heads, and replaces RoI Pooling with **RoI Align** to fix misalignment caused by quantization.

12.2 RoI Align — Why It Matters

RoI Pooling rounds proposal coordinates to integer grid cells, introducing misalignment between the RoI and extracted features. RoI Align instead samples feature values at exact (floating-point) locations using bilinear interpolation:

$$f(x, y) = \sum_{i,j} f(x_i, y_j) \max(0, 1 - |x - x_i|) \max(0, 1 - |y - y_j|)$$

This sub-pixel accuracy is essential for producing crisp masks.

12.3 Multi-Task Loss

$$\mathcal{L} = \mathcal{L}_{cls} + \mathcal{L}_{box} + \mathcal{L}_{mask}$$

$\mathcal{L}_{mask}$ is the average binary cross-entropy over the $m \times m$ mask output, computed **only** for the ground-truth class (masks for other classes don't contribute, avoiding competition between classes).

12.4 Full Code (PyTorch, torchvision)

```
import torch
from torchvision.models.detection import maskrcnn_resnet50_fpn
from torchvision.models.detection.faster_rcnn import FastRCNNPredictor
from torchvision.models.detection.mask_rcnn import MaskRCNNPredictor

model = maskrcnn_resnet50_fpn(weights="DEFAULT")

num_classes = 6
# box head
in_features = model.roi_heads.box_predictor.cls_score.in_features
model.roi_heads.box_predictor = FastRCNNPredictor(in_features, num_classes)
# mask head
in_features_mask = model.roi_heads.mask_predictor.conv5_mask.in_channels
model.roi_heads.mask_predictor = MaskRCNNPredictor(in_features_mask, 256, num_classes)

model.eval()
image = torch.rand(3, 480, 640)
with torch.no_grad():
    out = model([image])

print(out[0].keys())          # dict_keys(['boxes', 'labels', 'scores', 'masks'])
print(out[0]["masks"].shape)  # [N, 1, H, W] per-instance binary masks
```

13. YOLO — You Only Look Once (2016)

13.1 Definition

YOLO (Redmon et al.) reframes detection as a **single regression problem**: one CNN forward pass directly predicts all bounding boxes and class probabilities for the whole image, in one shot — no separate region proposal stage. This makes YOLO dramatically faster than R-CNN-family two-stage detectors, suitable for real-time detection.

13.2 Grid-Based Prediction

The input image is divided into an $S \times S$ grid. Each grid cell predicts B bounding boxes (each with x, y, w, h and a confidence score) plus C class probabilities:

$$\text{output tensor shape} = S \times S \times (B \cdot 5 + C)$$

The confidence score is defined as:

$$\text{conf} = \Pr(\text{object}) \times \text{IoU}(\text{pred}, \text{truth})$$

13.3 Intersection over Union (IoU)

$$\text{IoU} = \frac{\text{Area}(B_{pred} \cap B_{gt})}{\text{Area}(B_{pred} \cup B_{gt})}$$

IoU is used both as a training target (confidence) and at inference for **Non-Max Suppression (NMS)**, which removes duplicate overlapping boxes above an IoU threshold.

13.4 YOLO Loss (sum-of-squares, simplified)

$$\mathcal{L} = \lambda_{coord} \sum_{i,j} \mathbb{1}_{ij}^{obj} \Big[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 + (\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2 \Big]$$

$$+ \sum_{i,j} \mathbb{1}_{ij}^{obj} (C_i - \hat{C}_i)^2 + \lambda_{noobj} \sum_{i,j} \mathbb{1}_{ij}^{noobj} (C_i - \hat{C}_i)^2 + \sum_{i} \mathbb{1}_{i}^{obj} \sum_{c} (p_i(c) - \hat{p}_i(c))^2$$

Square roots on w, h de-emphasize errors on large boxes relative to small ones.

13.5 Full Code (PyTorch) — Simplified YOLO-style Head

```
import torch
import torch.nn as nn

class YOLOHead(nn.Module):
    """A minimal YOLO-style detection head on top of any CNN backbone."""
    def __init__(self, in_c, S=7, B=2, C=20):
        super().__init__()
        self.S, self.B, self.C = S, B, C
        self.conv = nn.Sequential(
            nn.Conv2d(in_c, 1024, 3, padding=1), nn.LeakyReLU(0.1),
            nn.Conv2d(1024, 1024, 3, padding=1), nn.LeakyReLU(0.1),
        )
        self.fc = nn.Sequential(
            nn.Flatten(),
```

```
            nn.Linear(1024 * S * S, 4096), nn.LeakyReLU(0.1), nn.Dropout(0.5),
            nn.Linear(4096, S * S * (B * 5 + C)),
        )

    def forward(self, x):
        x = self.conv(x)
        x = self.fc(x)
        return x.view(-1, self.S, self.S, self.B * 5 + self.C)

# Example: backbone produces a 1024-channel, 7x7 feature map
backbone_features = torch.randn(1, 1024, 7, 7)
head = YOLOHead(in_c=1024, S=7, B=2, C=20)
out = head(backbone_features)
print(out.shape)  # torch.Size([1, 7, 7, 30])  -> 2*5 + 20 = 30 values per cell

# Using a real pretrained YOLO (Ultralytics) for inference:
# pip install ultralytics
# from ultralytics import YOLO
# model = YOLO("yolov8n.pt")
# results = model("image.jpg")
# results[0].show()
```

14. RetinaNet (2017)

14.1 Definition

RetinaNet (Lin et al.) is a single-stage detector (like YOLO/SSD) that matches two-stage accuracy by solving the **class imbalance** problem: in a dense, single-shot detector almost all candidate locations are easy background, which overwhelms gradient updates. RetinaNet's fix is the **Focal Loss**, combined with a **Feature Pyramid Network (FPN)** backbone for strong multi-scale features.

14.2 Focal Loss — Math

Standard cross-entropy treats easy and hard examples equally:

$$CE(p_t) = -\log(p_t)$$

Focal loss adds a modulating factor $(1 - p_t)^\gamma$ that **down-weights well-classified examples**, focusing training on hard ones:

$$FL(p_t) = -\alpha_t (1 - p_t)^\gamma \log(p_t)$$

where p_t is the model's estimated probability for the true class, $\gamma \geq 0$ (typically 2) is the focusing parameter, and α_t balances class frequency. When $\gamma = 0$, FL reduces exactly to CE. As $p_t \to 1$ (easy, correctly-classified example), $(1-p_t)^\gamma \to 0$ and the loss contribution vanishes — exactly what's needed when 100,000+ background anchors would otherwise drown out the few foreground ones.

14.3 Feature Pyramid Network (FPN)

FPN builds a top-down pathway with lateral connections so that **every** pyramid level (from high-resolution/fine to low-resolution/coarse) carries strong semantic features, letting RetinaNet detect both small and large objects from the same backbone pass.

14.4 Full Code (PyTorch)

```
import torch
import torch.nn as nn

class FocalLoss(nn.Module):
    def __init__(self, alpha=0.25, gamma=2.0):
        super().__init__()
        self.alpha, self.gamma = alpha, gamma

    def forward(self, logits, targets):
        p = torch.sigmoid(logits)
        ce = nn.functional.binary_cross_entropy_with_logits(logits, targets, reduction="none")
        p_t = p * targets + (1 - p) * (1 - targets)
        alpha_t = self.alpha * targets + (1 - self.alpha) * (1 - targets)
        loss = alpha_t * (1 - p_t) ** self.gamma * ce
        return loss.mean()

# quick sanity check: focal loss down-weights easy (high p_t) examples
logits = torch.tensor([4.0, -4.0, 0.1])   # easy-pos, easy-neg, hard
targets = torch.tensor([1.0, 0.0, 1.0])
print(FocalLoss()(logits, targets).item())
```

```
# Using torchvision's full pretrained RetinaNet for inference:
from torchvision.models.detection import retinanet_resnet50_fpn_v2
model = retinanet_resnet50_fpn_v2(weights="DEFAULT")
model.eval()
image = torch.rand(3, 480, 640)
with torch.no_grad():
    preds = model([image])
print(preds[0].keys())  # dict_keys(['boxes', 'scores', 'labels'])
```

15. Putting It All Together — Comparisons and Trends

15.1 Classification Backbones: Error Rate Over Time

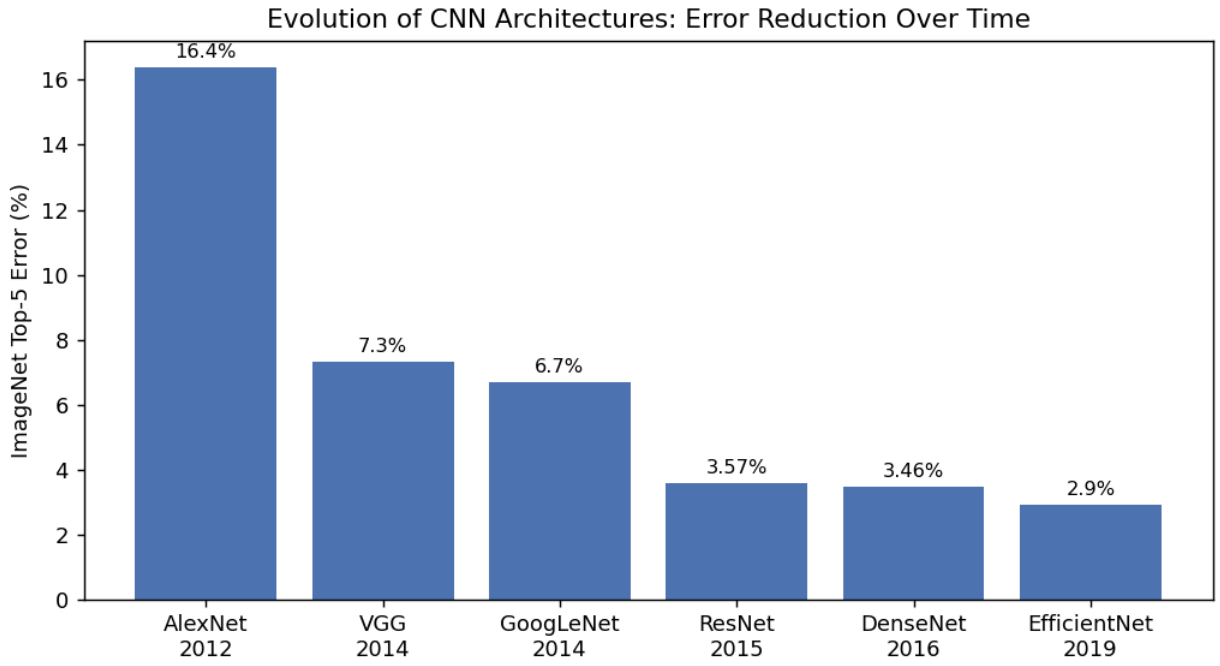

Figure 7: ImageNet top-5 error has dropped from 16.4% (AlexNet, 2012) to under 3% (EfficientNet, 2019) as architectures got deeper and smarter rather than just bigger.

15.2 Parameter Efficiency

Depth alone is not the goal — modern architectures achieve *better* accuracy with *fewer* parameters by using smarter building blocks (residuals, dense connections, depthwise separable convolutions, compound scaling).

15.3 Summary Table — Classification Backbones

Network	Year	Depth	Params	Key Idea
LeNet-5	1998	7	60K	First practical CNN
AlexNet	2012	8	60M	ReLU + Dropout + GPU training
VGG-16	2014	16	138M	Deep stacks of 3x3 convs
GoogLeNet	2014	22	5M	Multi-scale Inception modules
ResNet-50	2015	50	25M	Residual / skip connections
DenseNet-121	2016	121	8M	Dense feature concatenation
MobileNet	2017	28	4.2M	Depthwise separable convolutions

Network	Year	Depth	Params	Key Idea
EfficientNet-B0	2019	varies	5.3M	Compound scaling + MBConv + SE

15.4 Summary Table — Dense Prediction / Detection Networks

Network	Task	Stage(s)	Key Idea
U-Net	Semantic Segmentation	Single (encoder-decoder)	Skip connections preserve spatial detail
Faster R-CNN	Object Detection	Two-stage	Region Proposal Network replaces external proposals
Mask R-CNN	Instance Segmentation	Two-stage	RoI Align + parallel mask head
YOLO	Object Detection	Single-stage	Whole image -> grid regression in one pass
RetinaNet	Object Detection	Single-stage	Focal loss solves foreground/background imbalance

15.5 How to Choose

- **Need maximum accuracy, compute is not a constraint** → ResNet/DenseNet variants or EfficientNet as a classification backbone.
- **Need to run on a phone or embedded device** → MobileNet, EfficientNet- Lite.
- **Need pixel-level masks (medical imaging, satellite imagery)** → U-Net or Mask R-CNN.
- **Need real-time detection (video, robotics)** → YOLO.
- **Need the best accuracy/speed balance for detection with multi-scale objects** → RetinaNet or Faster R-CNN with an FPN backbone.

15.6 Closing Note

Every architecture in this guide is, at its core, the same building block from Section 1 — convolution, pooling, and non-linearity — arranged with a new architectural idea: depth (VGG), multi-scale parallelism (Inception), skip connections (ResNet/U-Net), dense reuse (DenseNet), computational efficiency (MobileNet/EfficientNet), or task reformulation (YOLO, Faster/Mask R-CNN, RetinaNet). Understanding the convolution operation and the residual connection deeply will let you read the architecture diagram of nearly any modern vision model.

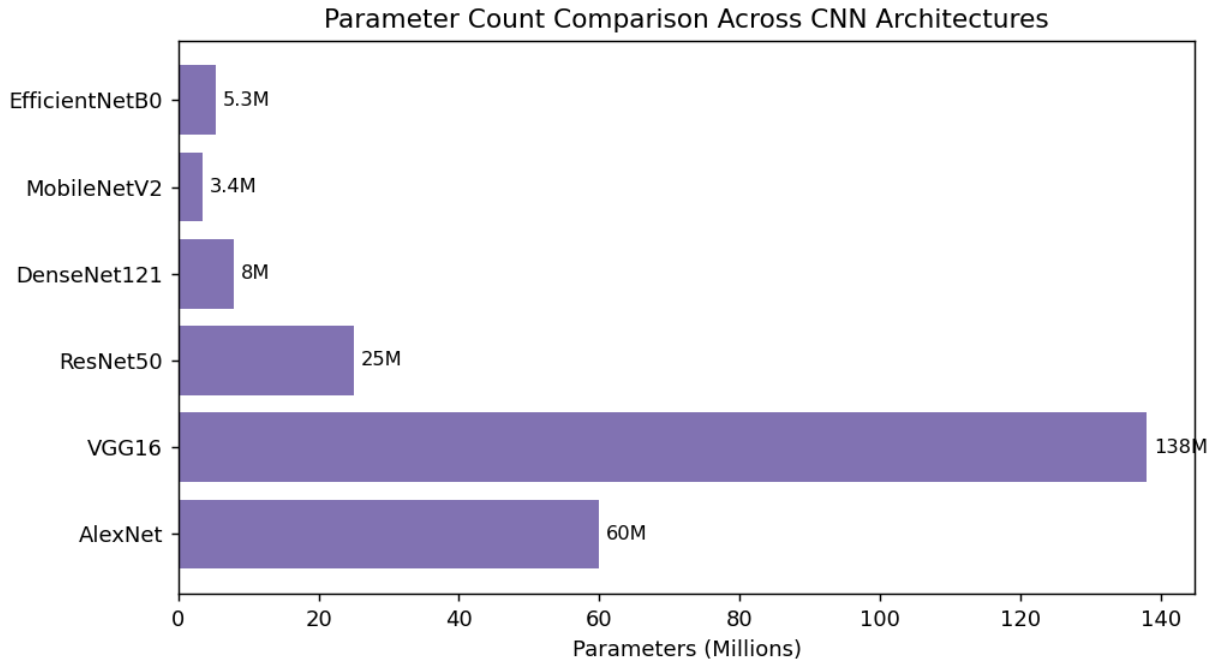

Figure 8: Parameter counts across architectures. Note how ResNet50, DenseNet121, MobileNetV2, and EfficientNetB0 achieve competitive or superior accuracy to VGG16 with a fraction of the parameters.